INDEX

SR NO.	DATE	TITLE	PAGE NO.	SIGN
1		To Study And Implement Substitution Cipher Transportation Cipher.		
2		To Study And Implement The RSA Algorithm For Public-Key Encryption And Decryption And Explore Its Properties And Security Considerations.		
3		To Study And Implement The Message Authentication Code For Ensuring The Message Intergrity And Authentication.		
4		Digital signature implement digital signature algorithm such as RSA based signature and verify the intergrity and authenticity of digital signature message algorithm.		
5		A Key Exchange Using Diffine Hellmen Implement The Diffine Hellman Key Exchange Algorithm To Security Exchange Key Between Two Entities Over An Insecure Network.		
6		To Configure IPSEC On Network Devices To Provide Secure Communication And Protect Against Unauthorized Access And Attacks.		
7		Web Security With SSL/TLS: Configure And Implement Secure Web Communication Using SSL/TLS Protocols Including Certificate Management And Secure Session Establishment.		
8		Intrusion Detection System: Setup And Configure An Intrusion Detection (IDS) To Monitor Network Traffic And Detact Potential Security Breaches Or Malicious Activities.		
9		Malware Analysis and Detection: Analyze and Identity malware samples using antivirus tools, analyze their behavior and develop counter measures to mitigate their impact.		
10		Firewall Configuration and Rule-based Filtering: Configure and test firewall rules to control network traffic, filter packets based on specified criteria, and protect network resources from unauthorized access.		

PRACTICAL NO. 1

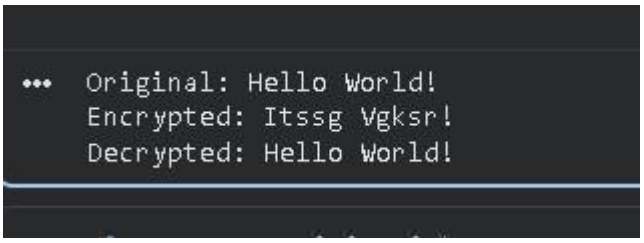
A. TO STUDY AND IMPLEMENT SUBSTITUTIOM CYPHER.

INPUT:-

```
class SubstitutionCipher: def
init_(self, key):
self.key = key def encrypt(self, plaintext):
encrypted_text = "" for
char in plaintext: if
char.isalpha(): if
char.islower():
encrypted_text += self.key[ord(char) -
ord('a')].lower() else: else:
encrypted_text += self.key[ord(char) - ord('A')].upper()
encrypted_text encrypted_text def += charreturn
decrypt(self,ciphertext):
decrypted_text = "" for char in
ciphertext: if char.isalpha(): if
char.islower(): decrypted_text
+= chr
(self.key.lower().index(char) + ord('a') ) else:
  decrypted_text +=
chr
( self.key.upper().index(char) + ord('A')
      else:
decrypted_text += char return
decrypted_text
key = "QWERTYUIOPASDFGHJKLZXCVBNM"
```

sub_cipher = SubstitutionCipher(key) message = "Hello World!" encrypted_message = sub_cipher.encrypt(message) decrypted_message = sub_cipher.decrypt(encrypted_message) print("Original:", message) print("Encrypted:", encrypted_message) print("Decrypted:", decrypted_message)

OUTPUT:-



B. TRANSPOSITION CYPHER.

INPUT:-

```
class TranspositionCipher: def
    init (self, key): self.key =
    key

    def encrypt(self, plaintext): num_columns
        = len(self.key)
        num_rows = len(plaintext) // num_columns + (
            1 if len(plaintext) % num_columns else 0
        )

        matrix = [["" for _ in range(num_columns)] for _ in range(num_rows)]

        for i, char in enumerate(plaintext): matrix[i //
        num_columns][i % num_columns] = char
        encrypted_text = ""
        for col in self.key:
            col_index = int(col) - 1 encrypted_text
            += "".join(
                matrix[row][col_index] for row in range(num_rows)
            )
return encrypted_text
```

```
    def decrypt(self, ciphertext): num_columns
        = len(self.key)
        num_rows = len(ciphertext) // num_columns + (
            1 if len(ciphertext) % num_columns else 0
        )

        matrix = [["" for _ in range(num_columns)] for _ in range(num_rows)] char_index

        = 0

        for col in self.key:
            col_index = int(col) - 1

            for row in range(num_rows):
                if char_index < len(ciphertext):
                    matrix[row][col_index] =
                    ciphertext[char_index] char_index += 1
decrypted_text = ""

        for row in range(num_rows):
            for col in range(num_columns):
        decrypted_text += matrix[row][col]
        return decrypted_text.rstrip()

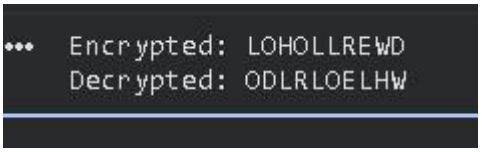

cipher = TranspositionCipher("3142") plaintext

= "HELLOWORLD"

encrypted = cipher.encrypt(plaintext)
print("Encrypted:", encrypted)

decrypted = cipher.decrypt(encrypted) print("Decrypted:",
decrypted)
```

OUTPUT:-



PRACTICAL NO. 2

A. TO STUDY AND IMPLEMENT THE RSA ALGORITHM FOR PUBLICKEY ENCRYPTION AND DECRYPTION, AND EXPLORE ITS PROPERTIES AND SECURITY CONSIDERATIONS.

INPUT:-

```
key = "4312567"
trans_cipher = TranspositionCipher(key)
message = "Hello World!" encrypted_message =
trans_cipher.encrypt(message)
decrypted_message = trans_cipher.decrypt(encrypted_message)
print("Original:", message) print("Encrypted:",
encrypted_message)
print("Decrypted:", decrypted_message)
```

OUTPUT:-

```
Original: Hello World!
Encrypted: ldllHoero! W
Decrypted: Hello orld!W
```

B.

INPUT :

```
import binascii
from Crypto.PublicKey import RSA from
Crypto.Cipher import PKCS1_OAEP
keypair = RSA.generate(2048) public_key = keypair.publickey()
print("RSA key pair generated successfully!\n") message = b"academy"
encryptor = PKCS1_OAEP.new(public_key) ciphertext
= encryptor.encrypt(message) print("Original message
:", message.decode())
print("Encrypted (hex) :", binascii.hexlify(ciphertext).decode())

decryptor = PKCS1_OAEP.new(keypair) plaintext
= decryptor.decrypt(ciphertext)

print("Decrypted message:", plaintext.decode())
```

```
RSA Key Pair Generated Successfully!

Original Message : Academy
Encrypted (Hex)  : 653daf0da3b3f050bc4cdcef92d0ab5899ae0d936313971e6d5ea7a3ac86f47a25f750b55382bf03383b24a6e2d6b0d58ae6710ef805a1f
Decrypted Message: Academy
```

PRACTICAL NO. 3

TO STUDY AND IMPLEMENT THE MESSAGE AUTHENTICATION CODE FOR ENSURING THE MESSAGE INTERGRITY AND AUTHENTICITY.

INPUT :-

```
import hashlib

result = hashlib.md5(b'Suyog')
result1 = hashlib.md5(b'Kunal')
result2 = hashlib.md5(b'Atharva')

print("The byte equivalent of hash is: ", end="") print(result.digest())

print("The byte equivalent of hash is: ", end="") print(result1.digest())

print("The byte equivalent of hash is: ", end="") print(result2.digest())
```

OUTPUT :-

```
Message: SHRAVANI
MD5 Hash: df720a531762342b84a213b1912ccb24

Message: SHIVAM
MD5 Hash: 24d2bcf69f8c20295a530c37820b8274

Message: YOJANA
MD5 Hash: 6e1dac5e8a8eacd902d1c8ff39b4c1c2
```

PRACTICAL NO. 4

DIGITAL SIGNATURE IMPLEMENT DIGITAL SIGNATURE ALGORITHM SUCH AS RSA BASED SIGNATURE AND VERIFY THE INTEGRITY AND AUTHENTICITY OF DIGITAL SIGNATURE MESSAGE ALGORITHM.

INPUT :-

```
!pip install -q pycryptodome

import binascii

from Crypto.PublicKey import RSA from
Crypto.Signature import pkcs1_15 from
Crypto.Hash import SHA256 key_pair =
RSA.generate(2048) private_key =
key_pair.export_key()

public_key = key_pair.publickey().export_key()
print("=" * 60)
print(" RSA DIGITAL SIGNATURE DEMONSTRATION")
print("=" * 60)
print("\nPublic Key:\n") print(public_key.decode())
print("\nPrivate Key:\n") print(private_key.decode())
message = "Hello, World!"
print("\nOriginal Message:") print(message)

hash_object = SHA256.new(message.encode()) private_key_obj =

RSA.import_key(private_key)

signature = pkcs1_15.new(private_key_obj).sign(hash_object)

print("\nDigital Signature (Hex):")

print(binascii.hexlify(signature).decode()) public_key_obj

= RSA.import_key(public_key) verify_hash =

SHA256.new(message.encode())

try:
    pkcs1_15.new(public_key_obj).verify(verify_hash, signature)
    print("\nSignature Verification: VALID") except
    (ValueError, TypeError):
    print("\nSignature Verification: INVALID")
```

OUTPUT:-

```
Original Message:
Hello, World!

Digital Signature (Hex):
81fc00d294a6bd74a9c8e09694ce2aa345a79cc2545260b0ec8ddc8839514349cd0513e72abf5947b9f9126a1a45ae71094953e33149a2637fa5782aac30e0ca93

Signature Verification: VALID
```

PRACTICAL NO. 5

A KEY EXCHANGE USING DIFFINE HELLMAN IMPLEMENT THE DIFFINE HELLMAN KEY EXCHANGE ALGORITHM TO SECURITY EXCHANGE KEY BETWEEN TWO ENTITIES OVER AN INSECURE NETWORK.

INPUT :-

```
import secrets

def diffie_hellman_exchange():
"""
Simulates the Diffie-Hellman Key Exchange between two parties: Alice and Bob. """

# 1. Publicly known variables
# Agreed upon over an insecure channel
# In real applications, P is a massive prime number.
# For this practical, we will use a smaller prime number.
P = 997 # Prime Modulus
G = 7   # Generator (Primitive root modulo P)

print("=== 1. Public Parameters (Visible to any eavesdropper) ===") print(f"Prime
Modulus (P): {P}")
print(f"Generator (G):     {G}\n")

# 2. Alice's Setup
# Alice generates her private key 'a'
# and calculates her public key 'A'
a = secrets.randbelow(P - 2) + 1
A = pow(G, a, P)

print("=== 2. Alice's Operations ===")
print(f"Alice's Private Key (a): {a} <-- (Kept completely secret)") print(f"Alice's
Public Key (A): {A} <-- (Sent to Bob over network)\n")

# 3. Bob's Setup
# Bob generates his private key 'b'
# and calculates his public key 'B'
b = secrets.randbelow(P - 2) + 1
B = pow(G, b, P)

print("=== 3. Bob's Operations ===") print(f"Bob's Private Key (b): {b}
<-- (Kept completely secret)") print(f"Bob's Public Key (B): {B} <--
(Sent to Alice over network)\n")

# 4. Exchange & Shared Secret Calculation
# Alice calculates the secret using Bob's public key
# and her private key
alice_shared_secret = pow(B, a, P)
```

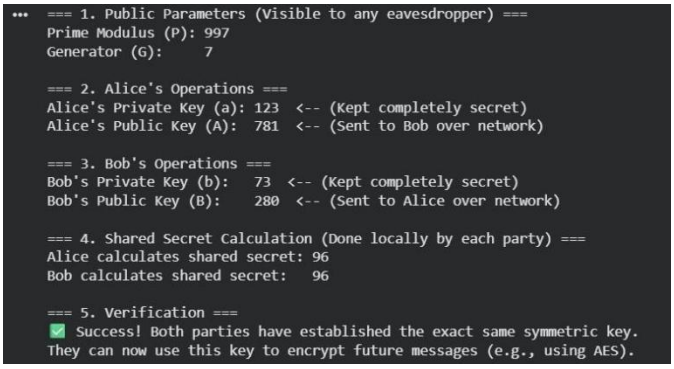
```
# Bob calculates the secret using Alice's public key
# and his private key
bob_shared_secret = pow(A, b, P)

print("=== 4. Shared Secret Calculation (Done locally by each party) ===")
print(f"Alice calculates shared secret: {alice_shared_secret}") print(f"Bob
calculates shared secret: {bob_shared_secret}\n")

# 5. Verification
print("=== 5. Verification ===")

if alice_shared_secret == bob_shared_secret: print("Success! Both parties have
    established the exact same symmetric key.") print("They can now use this key to
    encrypt future messages (e.g., using AES).")
else:
    print("Error! The keys do not match.")
```

OUTPUT :-

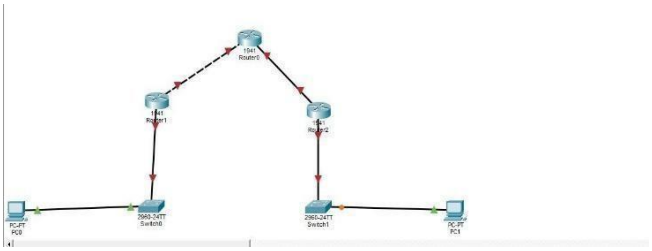


PRACTICAL NO. 6

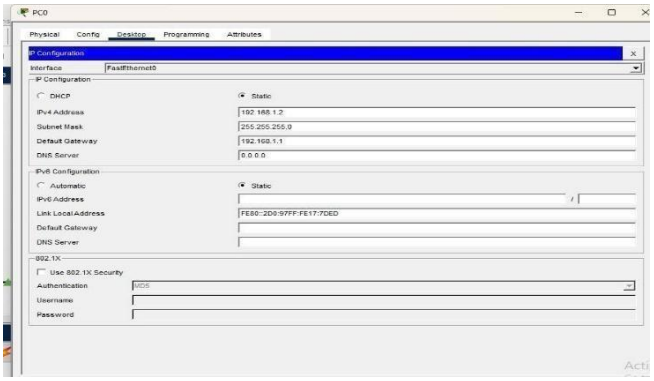
TO CONFIGURE IPSEC ON NETWORK DEVICES TO PROVIDE SECURE COMMUNICATION AND PROTECT AGAINST UNAUTHORIZED ACCESS AND ATTACKS.

INPUT :-

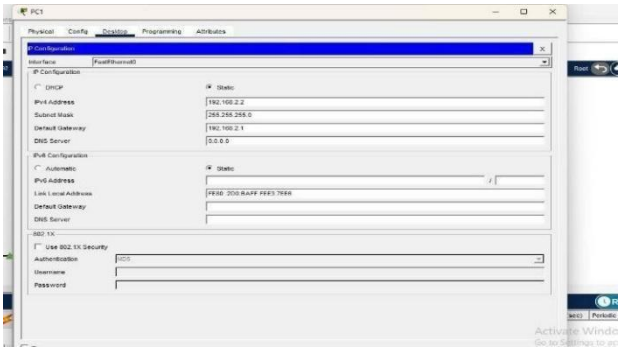
Step 1



Step 2

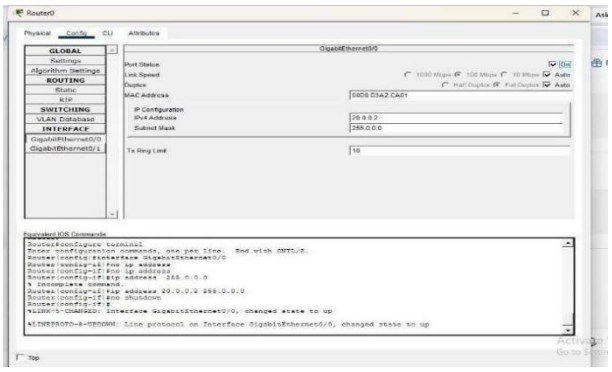


Step 3



Step

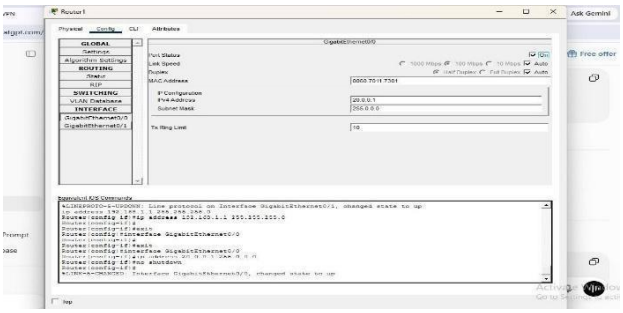
4



Step 5



Step 6



Step 7

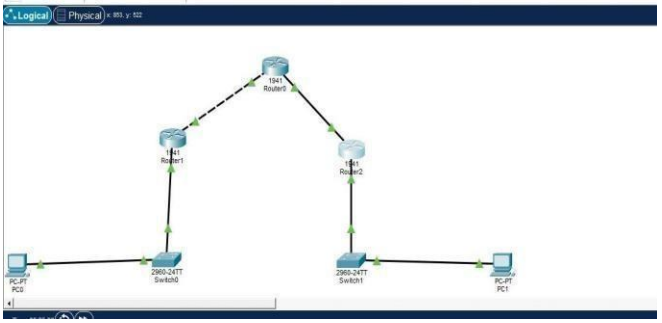


8

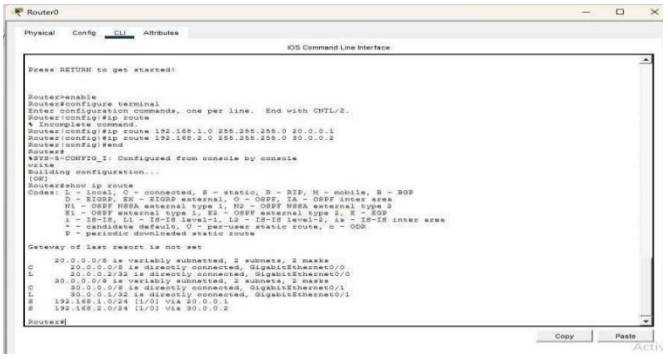
Step



Step 9



Step10



Step 11



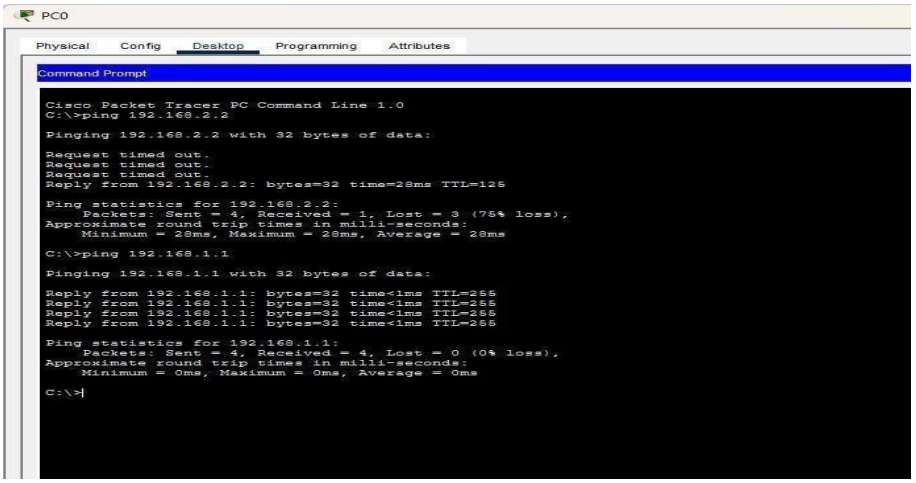
Step

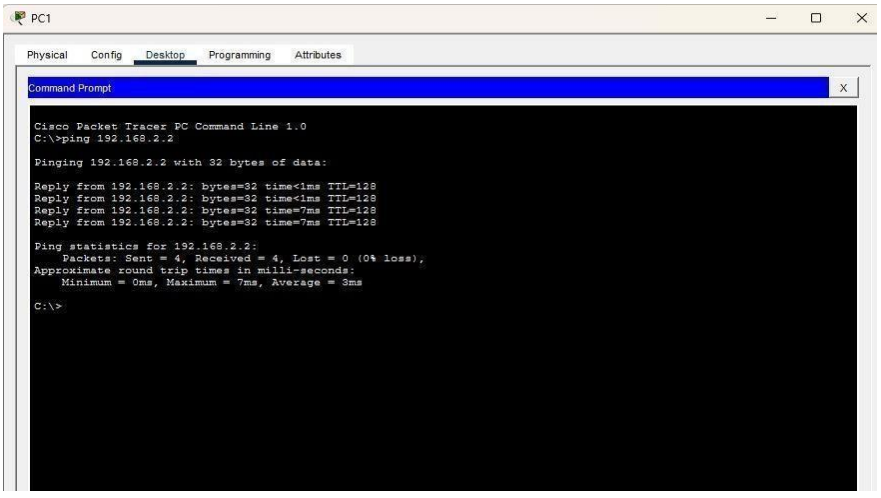
12



OUTPUT

:-





PRACTICAL NO. 7

WEB SECURITY WITH SSL/TLS: CONFIGURE AND IMPLEMENT SECURE WEB COMMUNICATION USING SSK/TLS PROTOCOLS, INCLUDING CERTIFICATE MANAGEMENT AND SECURE SESSION ESTABLISHMENT.

INPUT :-

```
import ssl import
socket

context = ssl.create_default_context()

sock = socket.create_connection(("example.com", 443)) ssl_sock =

context.wrap_socket(sock, server_hostname="example.com")

request = (
"GET / HTTP/1.1\r\n"
"Host: example.com\r\n"
"Connection: close\r\n"
"\r\n"
)

ssl_sock.sendall(request.encode())

response = b""

while True:
data = ssl_sock.recv(4096)

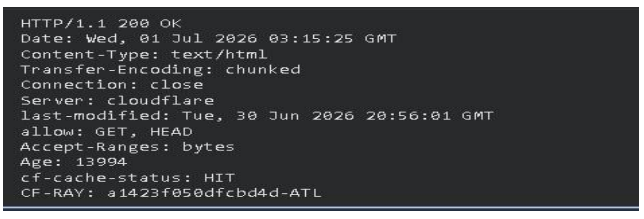
if not data: break

response += data ssl_sock.close() headers,

body = response.split(b"\r\n\r\n", 1)

print(headers.decode("utf-8"))
```

OUTPUT :-



PRACTICAL NO. 8

INTRUSION DETECTION SYSTEM SETUOP AND CONFIGURE AN INTRUSION SETECTION (IDS) TO MONITOR NETWORK TRAFFIC AND DETECT POTENTIAL SECURITY BREACHES OR MALICIOUS ACTIVITIES.

INPUT :-

```
import cv2 import numpy as np from
IPython.display import Image, display img =
cv2.imread("/content/shinchan.webp")

if img is None: print("Error:
Image not found.") exit()

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

thresh = cv2.threshold(
gray, 170, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)[1]

kernel = np.ones((10, 10), np.uint8)

eroded = cv2.erode(thresh, kernel, iterations=2) dilated

= cv2.dilate(eroded, kernel, iterations=2)

contours, _ = cv2.findContours(
dilated, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

intrusion_detected = False

for contour in contours: x, y, w, h =
cv2.boundingRect(contour)
aspect_ratio = float(w) / h area =
cv2.contourArea(contour)

    if aspect_ratio > 1.5 and area > 500:

        # Draw a green bounding box
        cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)
print( f"Intrusion detected at location: ({x}, {y}) "
        f"with width: {w} and height: {h}" )
        intrusion_detected = True

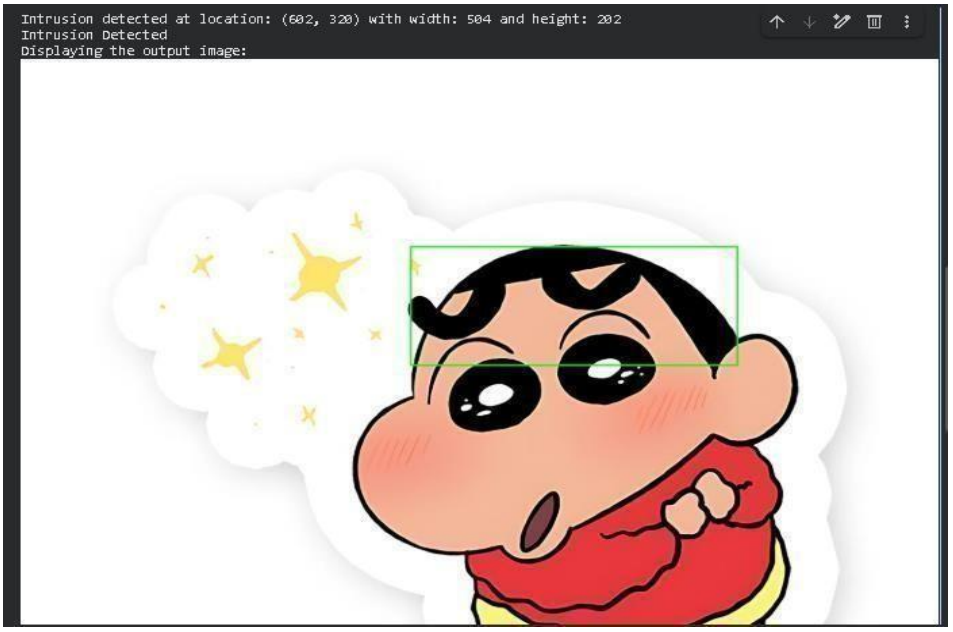

cv2.imwrite("output.jpg", img)

if intrusion_detected:
print("Intrusion Detected") else:
```

```
print("No intrusion detected.")

print("Displaying the output image:")
display(Image(filename="output.jpg"))
```

OUTPUT :-



PRACTICAL NO. 9

TO STUDY AND IMPLEMENT MALWARE ANALYSIS AND DETECTION USING ANTIVIRUS TOOLS, ANALYZE MALWARE BEHAVIOR, AND DEVELOP COUNTERMEASURES TO MITIGATE ITS IMPACT.

Step 1: Go to the website www.virusshare.com for downloading the clean sample of Malware



VirusShare.com - Because Sharing is Caring

Home • Hashes • Research • About • Swag Shop

Please login to search and download.

System currently contains 85,474,348 malware samples.

Report for a sample recently added to the system:
687092df8ea134514adc8295d069fbff53829da1d28b9b93343db3765468c039

VirusShare info last updated 2024-08-14 00:00:00 UTC

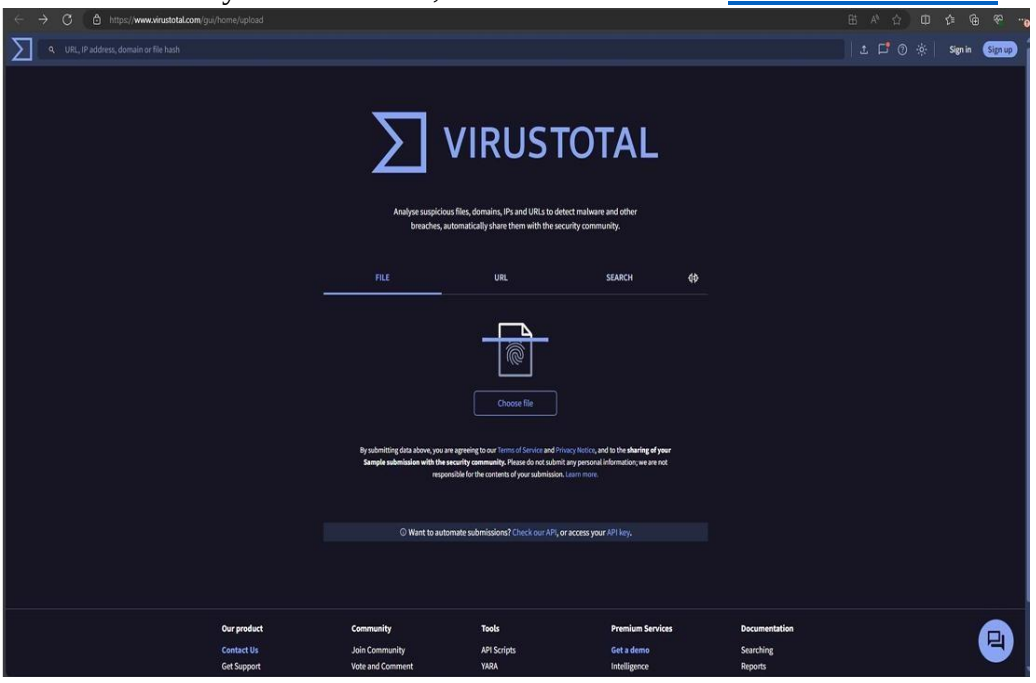
MD5	140a5776fd446abda2e48ea597029dc5
SHA1	8251b760974ece275b55dba0e720f7aa8d653ad7
SHA256	687092df8ea134514adc8295d069fbff53829da1d28b9b93343db3765468c039
SSDeep	98304:mRPjxx4EB/BQvDxMkhc9icWN0Xyb35LdFpp7O//1FeZRMWzrHAvAGzrvmp:mRjEEBe9Mk69cQ0iNxbp27ezbgIPp
Authentihash	1cf685379a937cce5c0f752977362b090a4b9a2ba176705c1c2b5a33db1719ec
Size	4,685,944 bytes
File Type	PE32 executable (GUI) Intel 80386, for MS Windows
Mime Type	application/x-dosexec
Extension	exe
TrID	Win32 Executable MS Visual C++ (generic) (37.8%) Microsoft Visual C++ compiled executable (generic) (20.0%) Win64 Executable (generic) (12.7%) Win32 Dynamic Link Library (generic) (7.9%) Win16 NE executable (generic) (6.1%)

Detections (65/75)	
ALYac	Trojan.GenericKD.67993768
APEX	Malicious
AVG	Win32:RansomX-gen [Ransom]
Acronis	suspicious
AhnLab-V3	Trojan/Win32.Cosmu.R51515
Antiy-AVL	Trojan/Win32.Cosmu
Arcabit	Trojan.Generic.D40D80A8
Avast	Win32:RansomX-gen [Ransom]
Avira	TR/ATRAPS.Gen
BitDefender	Trojan.GenericKD.67993768
BitDefenderTheta	Gen:NN.ZexaF.36810.@tZ@aGBV9uib
Bkav	W32.AIDetectMalware
CAT-QuickHeal	W32.Zombie.A4
ClamAV	Win.Malware.Generickdz-9938530-0
CrowdStrike	win/malicious_confidence_100% (W)
Cybereason	malicious.8fd446

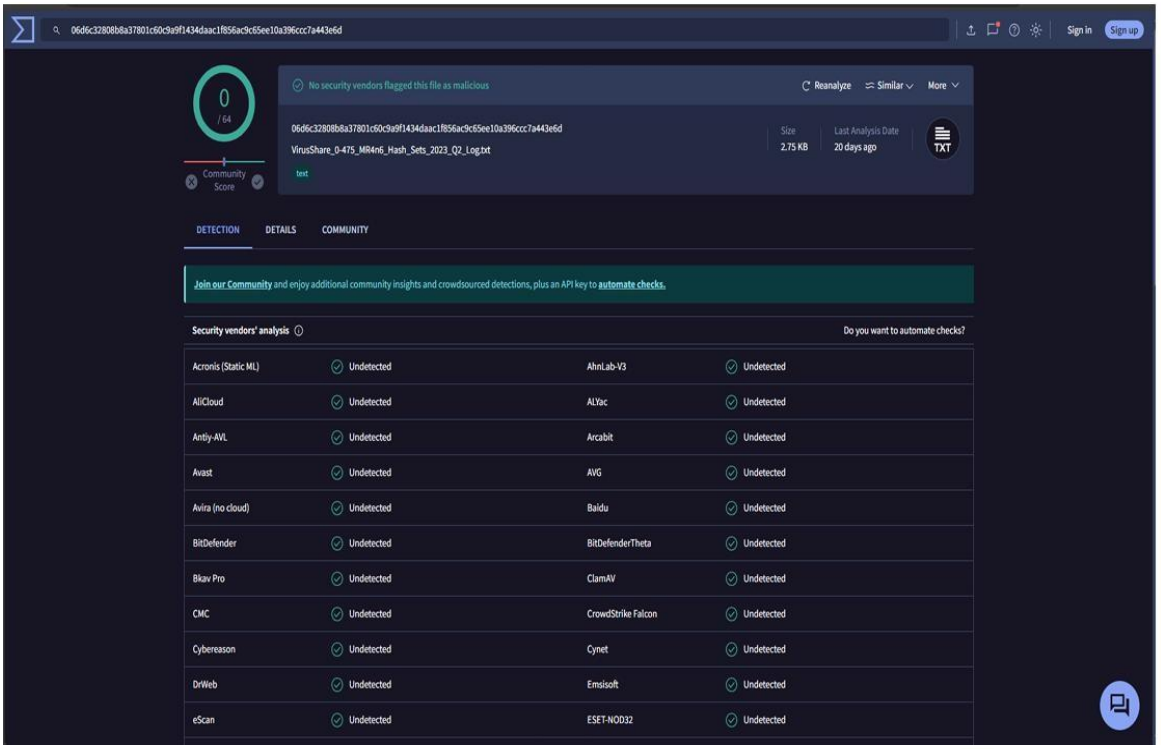
Step 2: By clicking the above download icon the Malware gets downloaded in ZIP format.

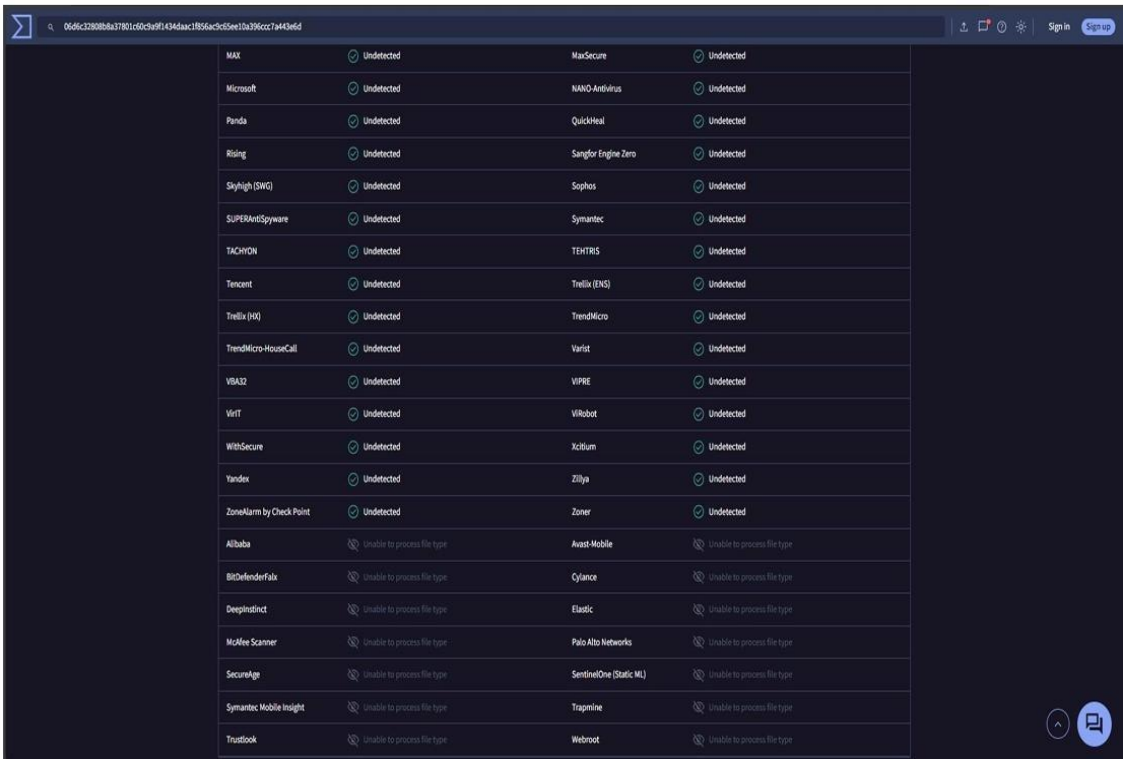


Step 3: In order to analyse the Malware, we select the website www.virustotal.com.



Step 4: Click on “Choose File” and select the file from the location.

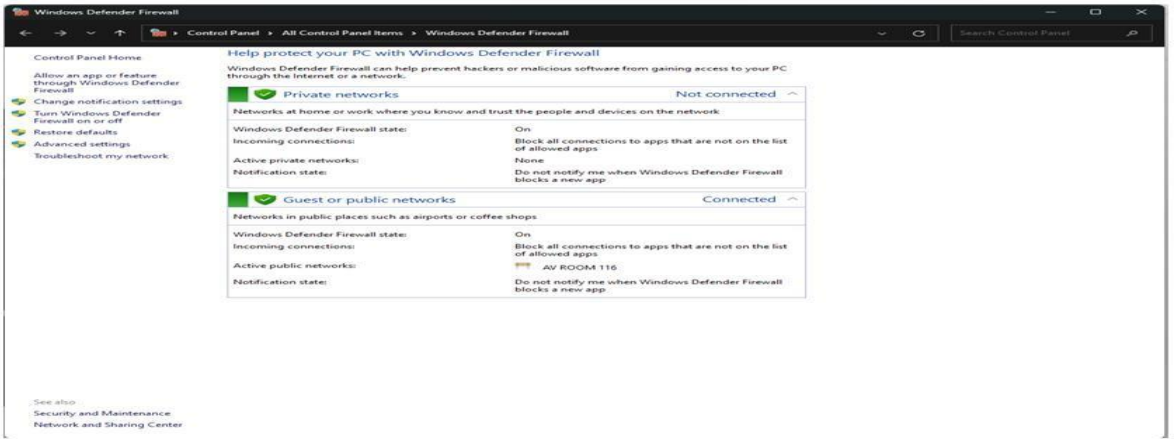




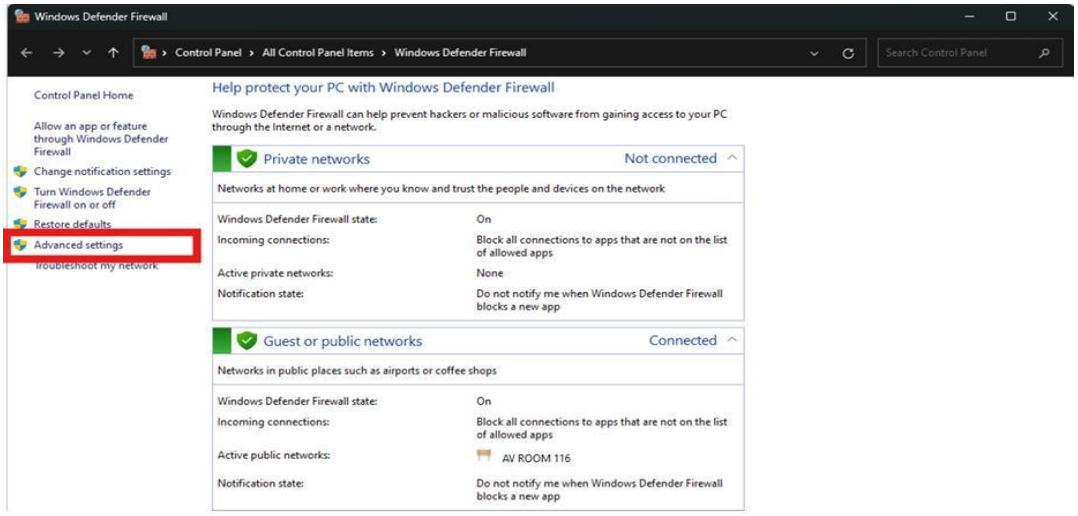
PRACTICAL NO. 10

FIREWALL CONFIGURATION AND RULE-BASED FILTERING: CONFIGURE AND TEST FIREWALL RULES TO CONTROL NETWORK TRAFFIC, FILTER PACKETS BASED ON SPECIFIED CRITERIA, AND PROTECT NETWORK RESOURCES FROM UNAUTHORIZED ACCESS.

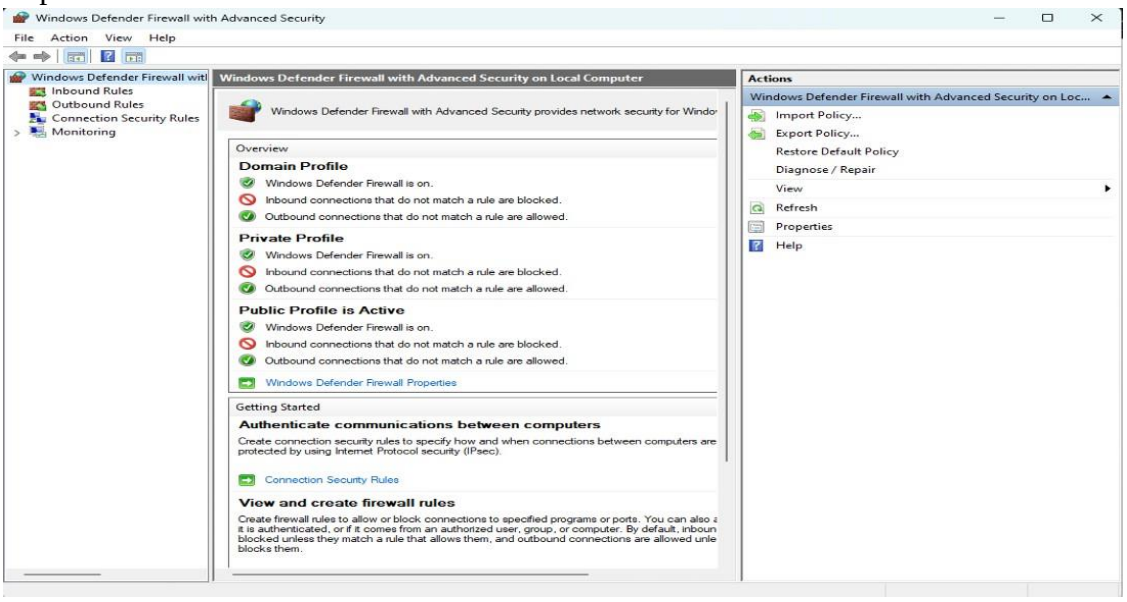
Step 1: Open ‘Windows Defender Firewall’.

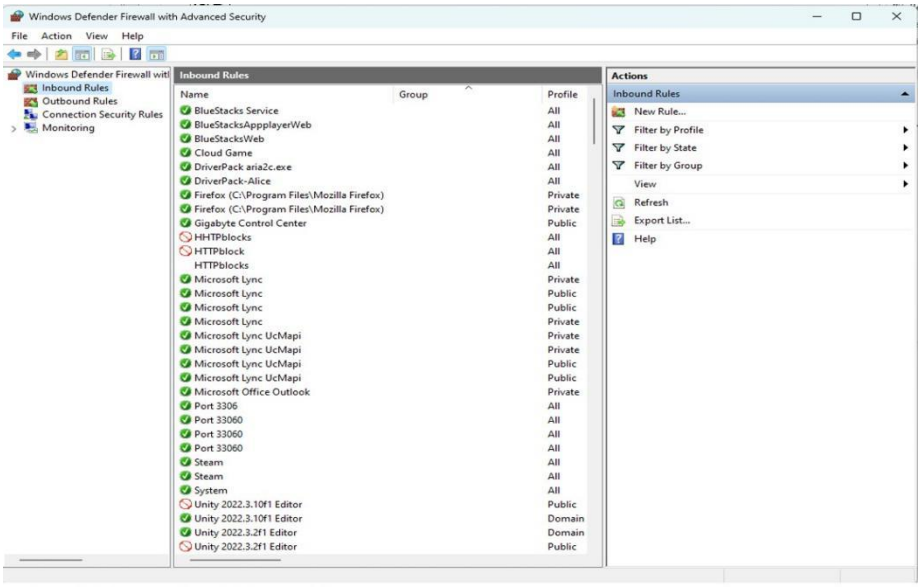


Step 2: Next we click on ‘Advanced settings.

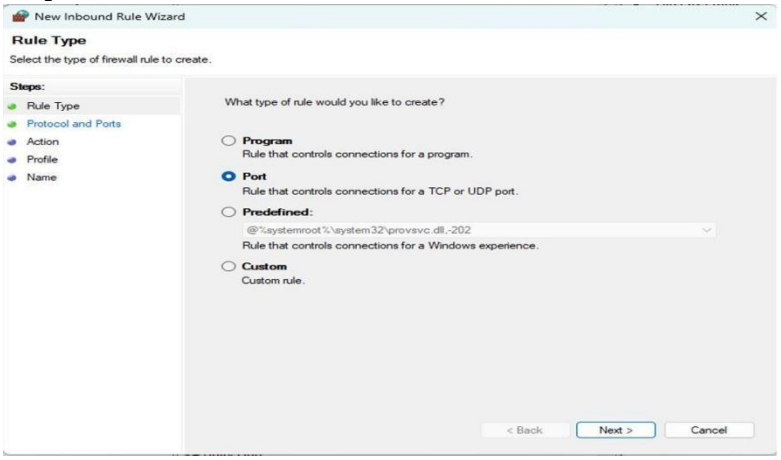


Step 3: Next we click on ‘Inbound Rules.

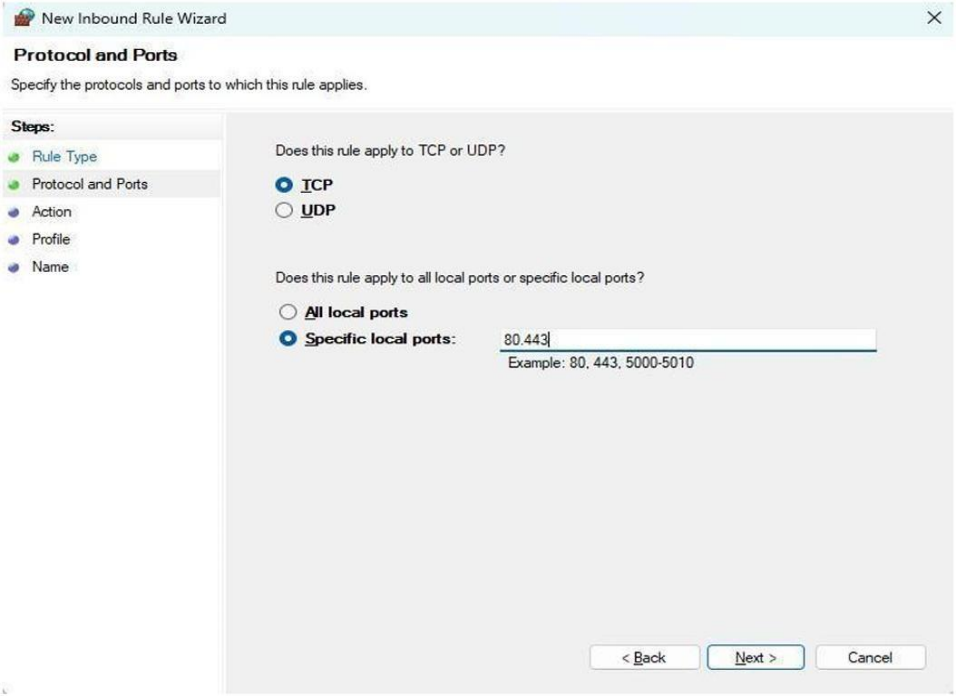




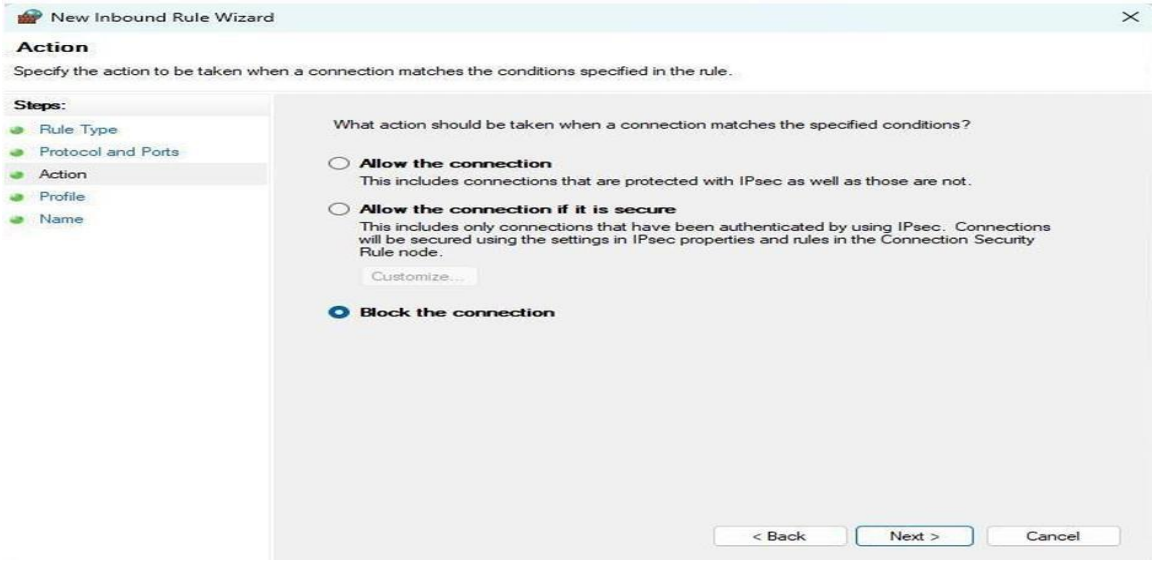
Step 4: Then click on ‘New Rule’



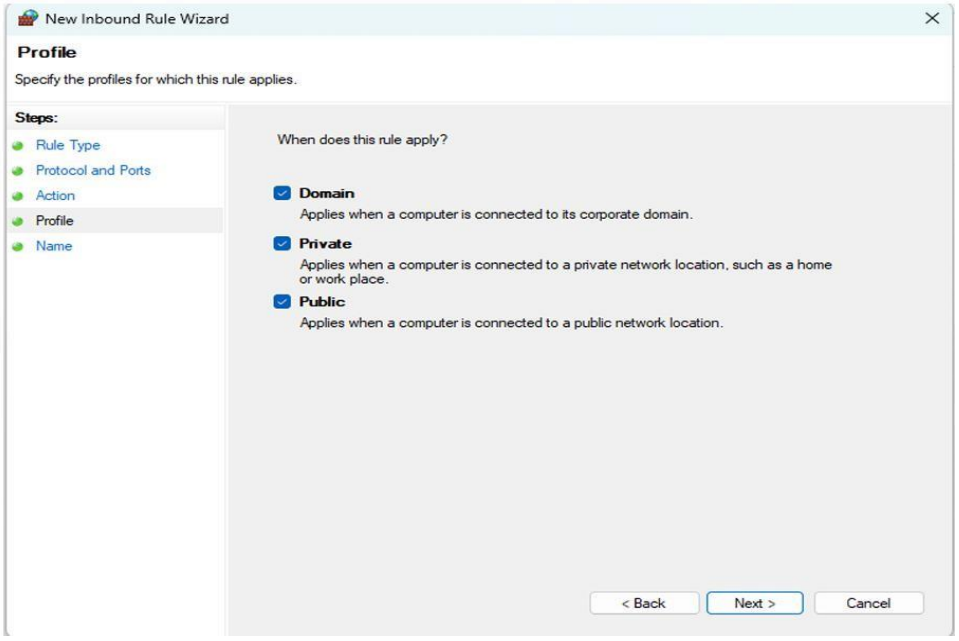
Step 5: Select the radio button ‘Port’ and click ‘Next’ and enter the following.



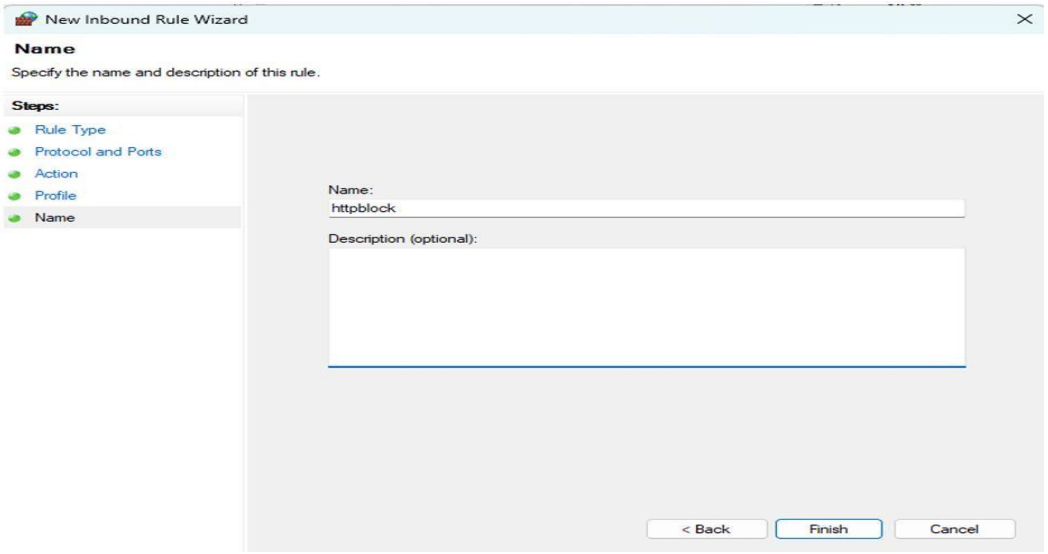
Step 6: After next, we need to finalise the rule



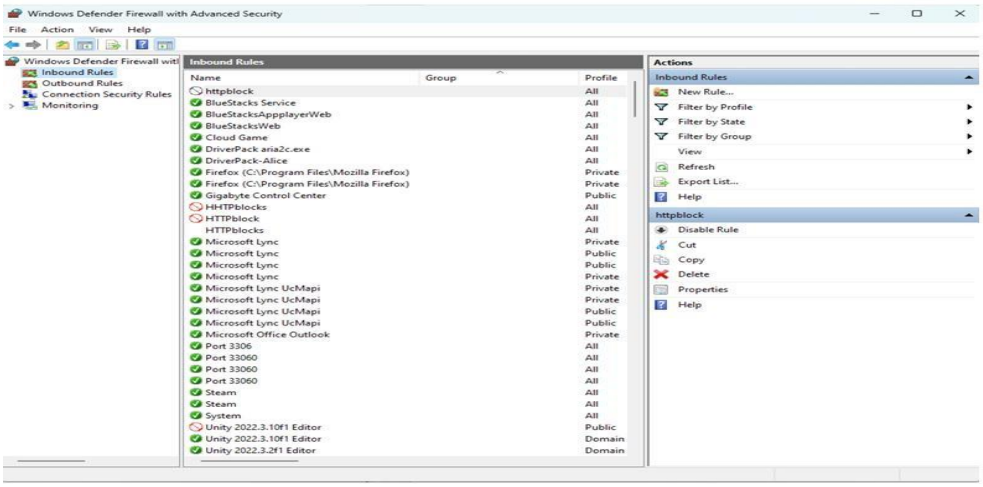
Step 7: Click 'Next' and we get the following



Step 8: After clicking the 'Next' button we need to name the rule and click finish



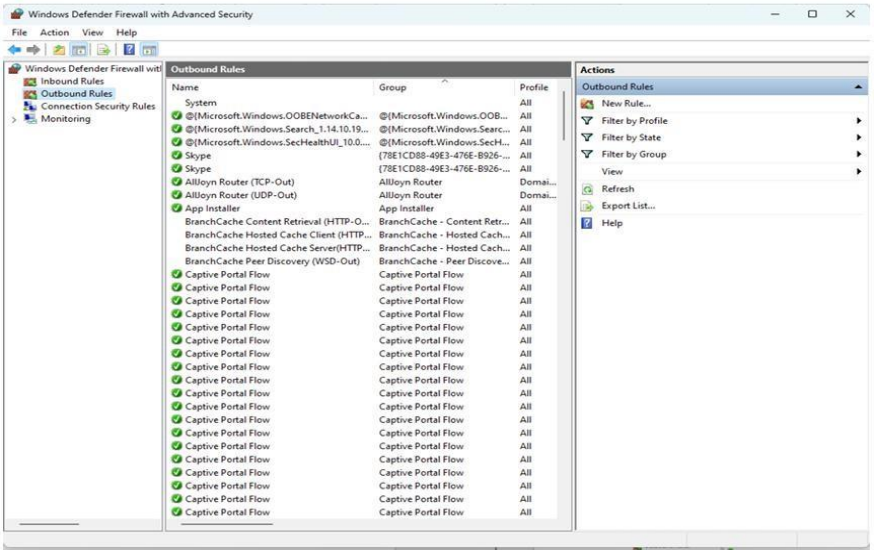
Step 9: The Inbound rule is added



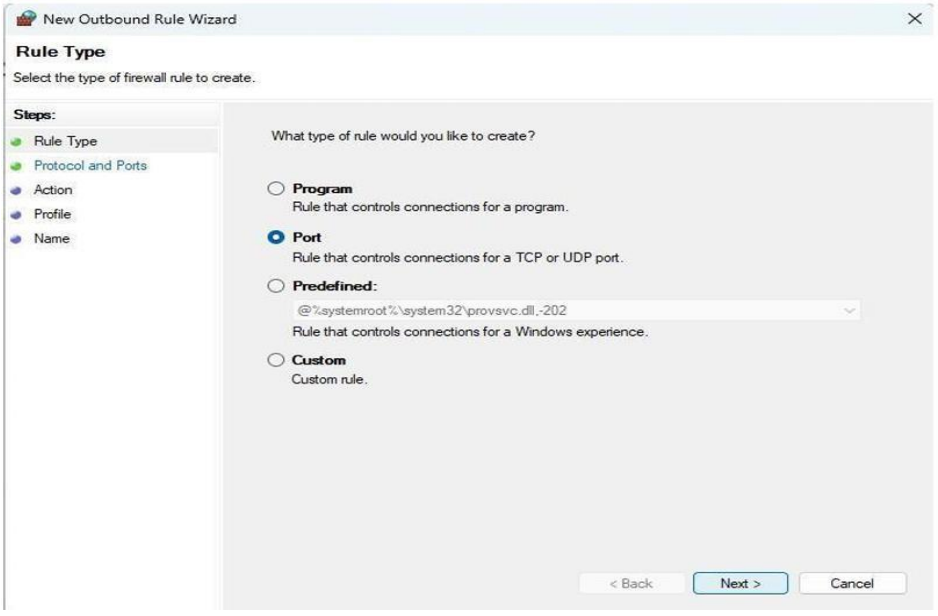
Same steps for Outbound rules

Step 1: Click on ‘Outbound Rules.

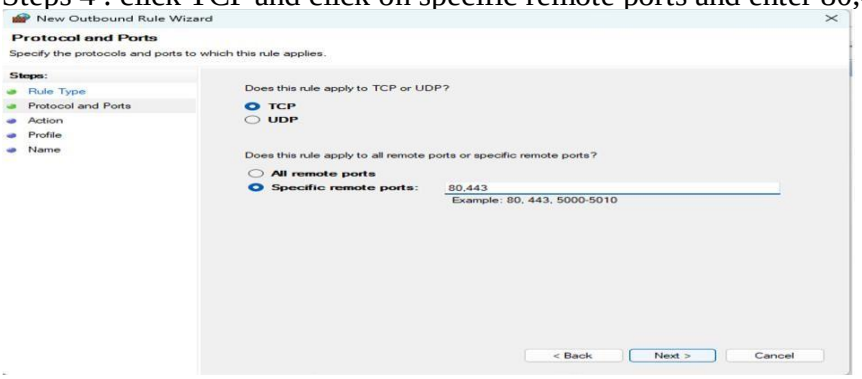
Step 2: Then click on ‘New Rule’ .



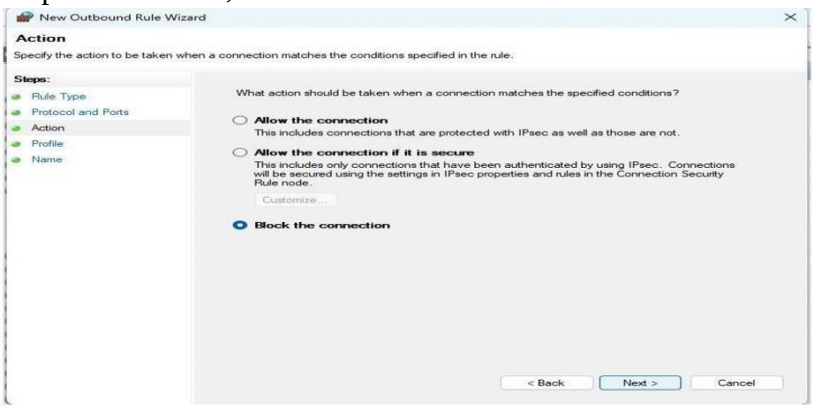
Step 3: Select the radio button ‘Port’ and click ‘Next’ and enter the following.



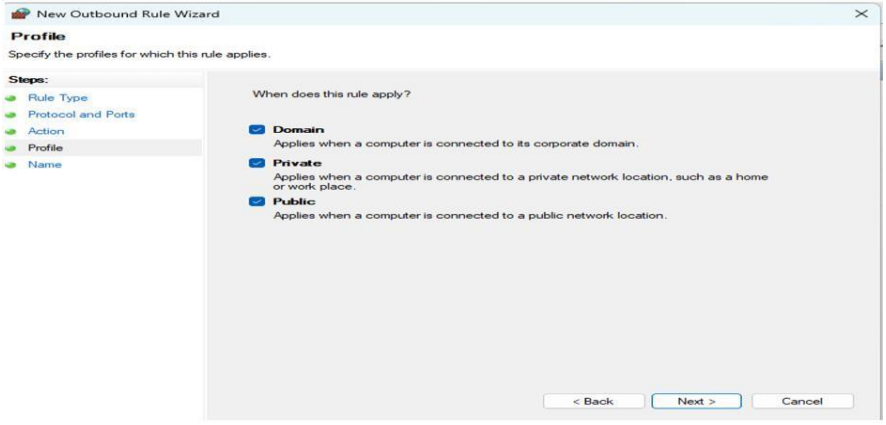
Steps 4 : click TCP and click on specific remote ports and enter 80,443.enter the Next .



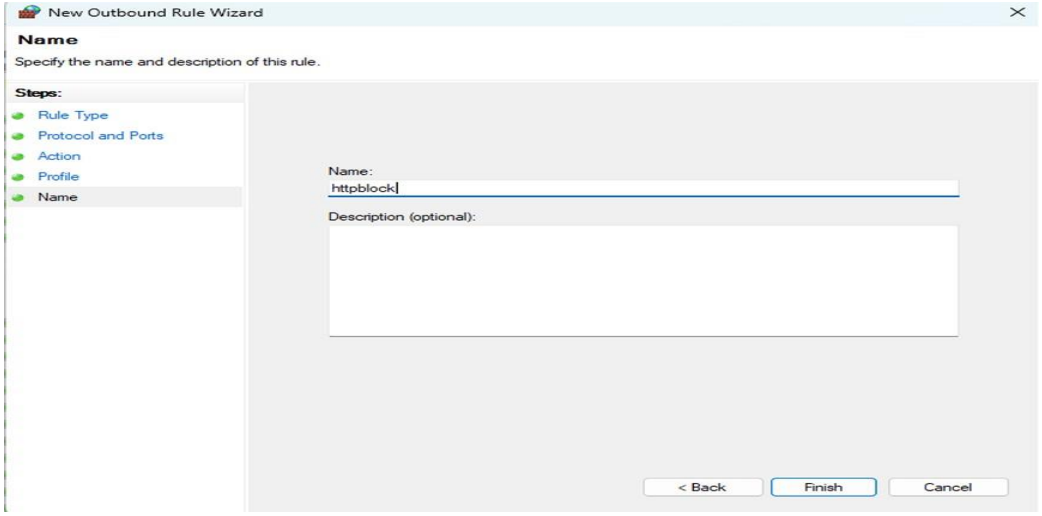
Step 5: After next, we need to finalise the rule



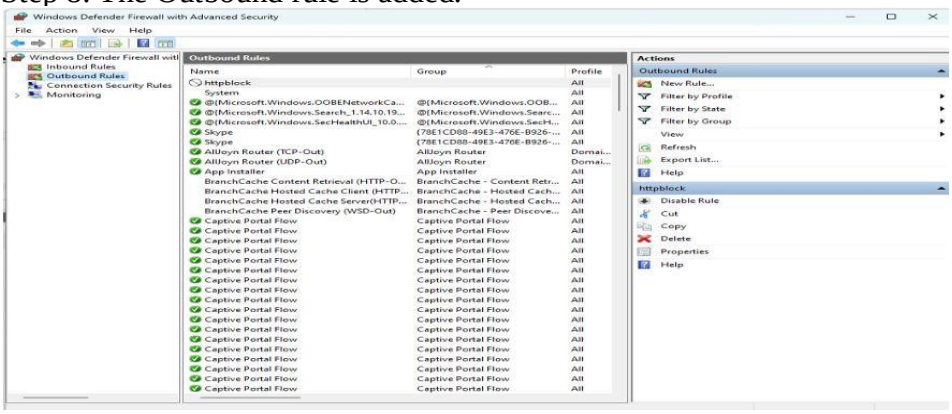
Step 6: Click ‘Next’ and we get the following



Step 7 : After clicking the ‘Next’ button we need to name the rule and click finish.



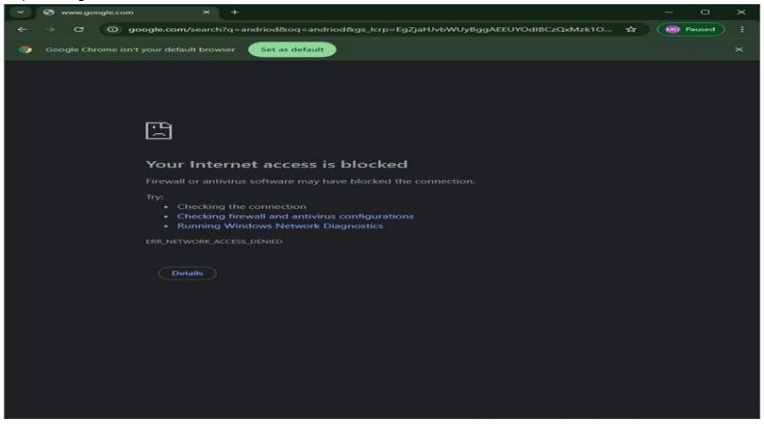
Step 8: The Outbound rule is added.



Step 8: The Outbound rule is added.

OUTPUT:

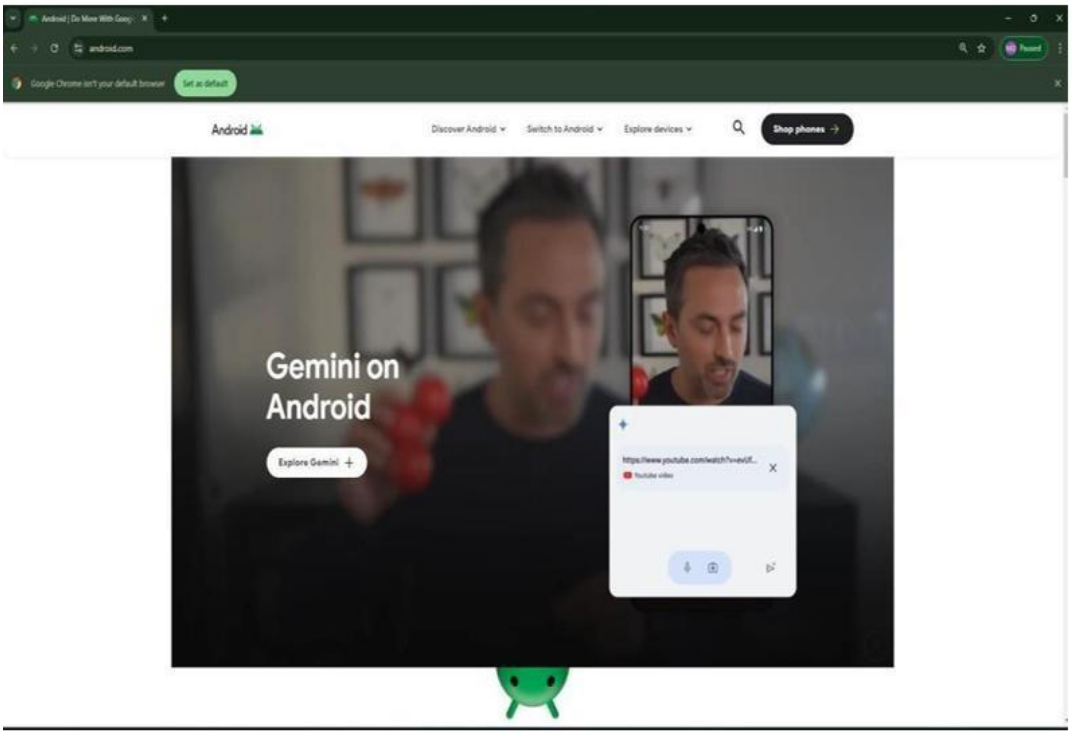
A) Try to access the internet. We see that the accessed is blocked.



B) BLOCKING SITE WWW.ANDRIOD.COM

INPUT:

Step 1: We open the browser and access the website, which is now accessible.



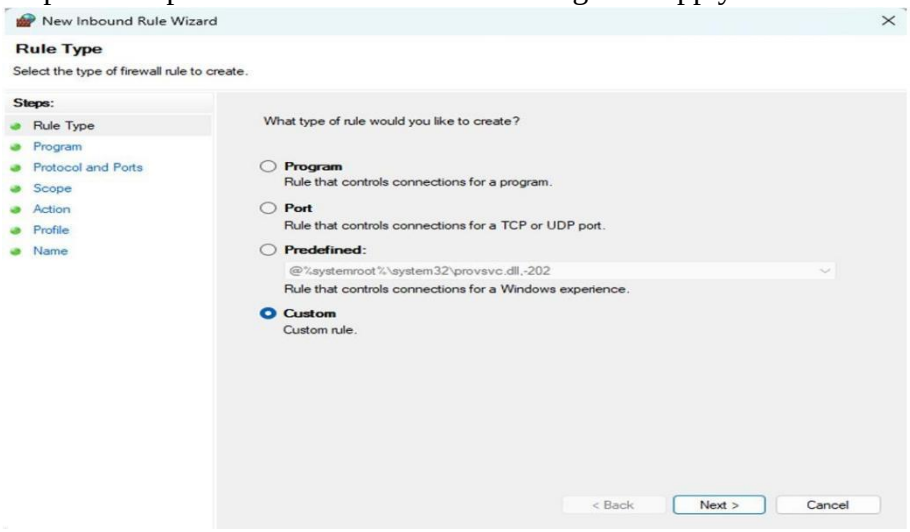
Step 2: We find the IP addresses of the website using the following command

```
C:\Users\admin>nslookup android.com
Server:  UnKnown
Address:  192.168.116.1

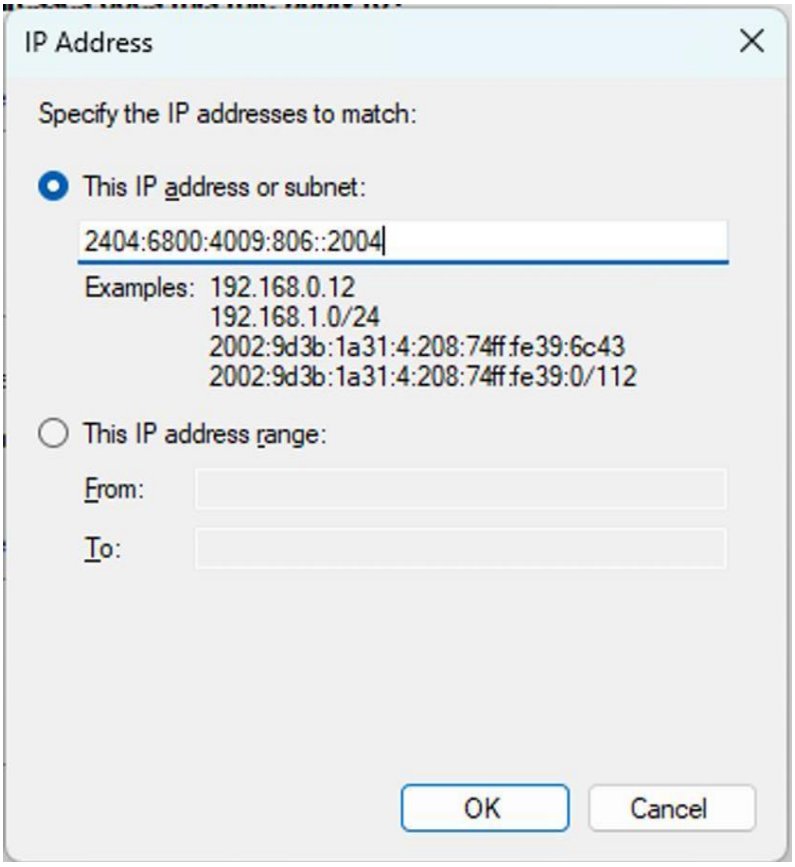
Non-authoritative answer:
Name:    android.com
Addresses:  2404:6800:4009:806::2004
          142.250.71.100

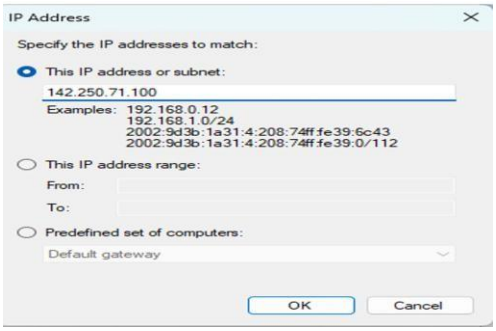

C:\Users\admin>
```

Step 3: We open the windows Firewall settings and apply the Inbound Rule

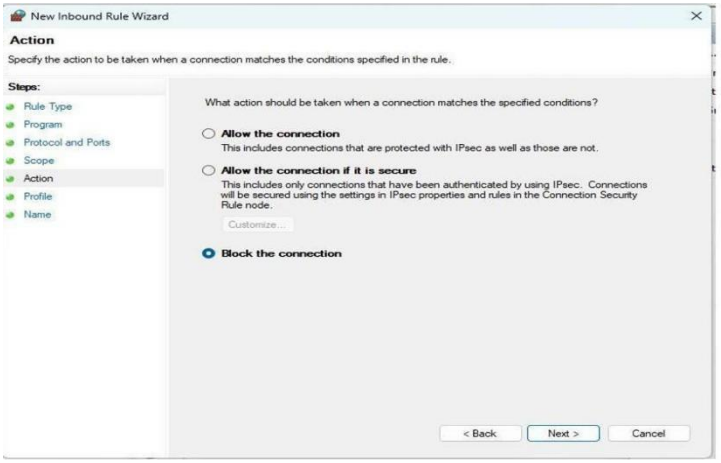


Step 4: Insert the IP addresses both IPv4 and IPv6.

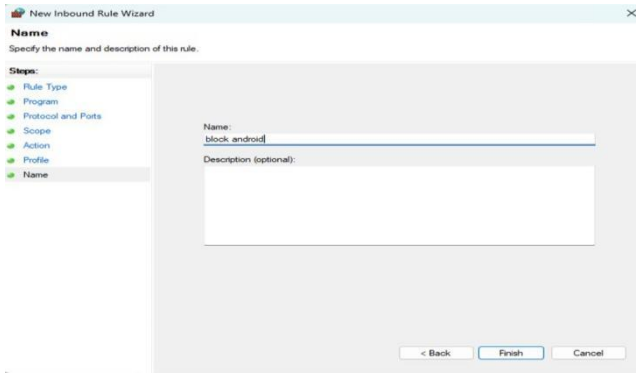




Step 5: Select Block connection

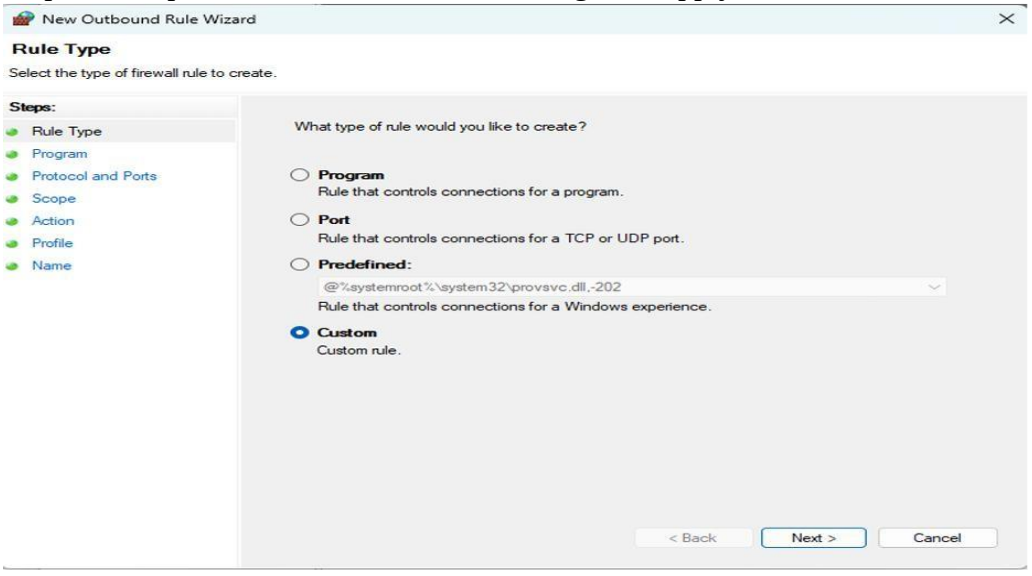


Step 6: Provide a suitable name and finish.

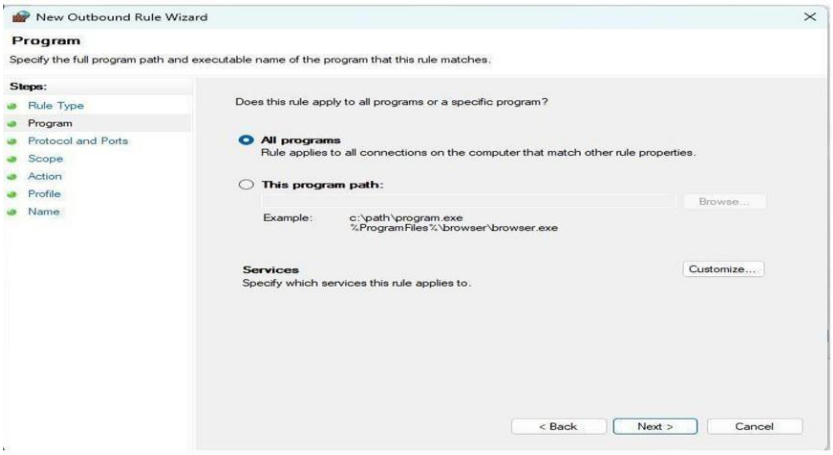


Repeat the above steps for Outbound Rules.

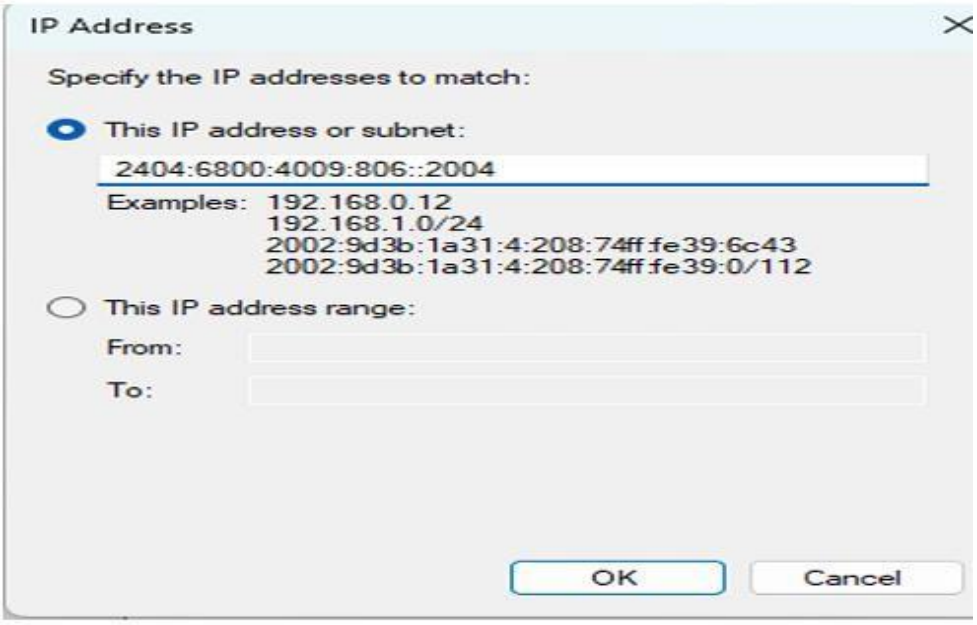
Step 1: We open the windows Firewall settings and apply the Inbound Rule

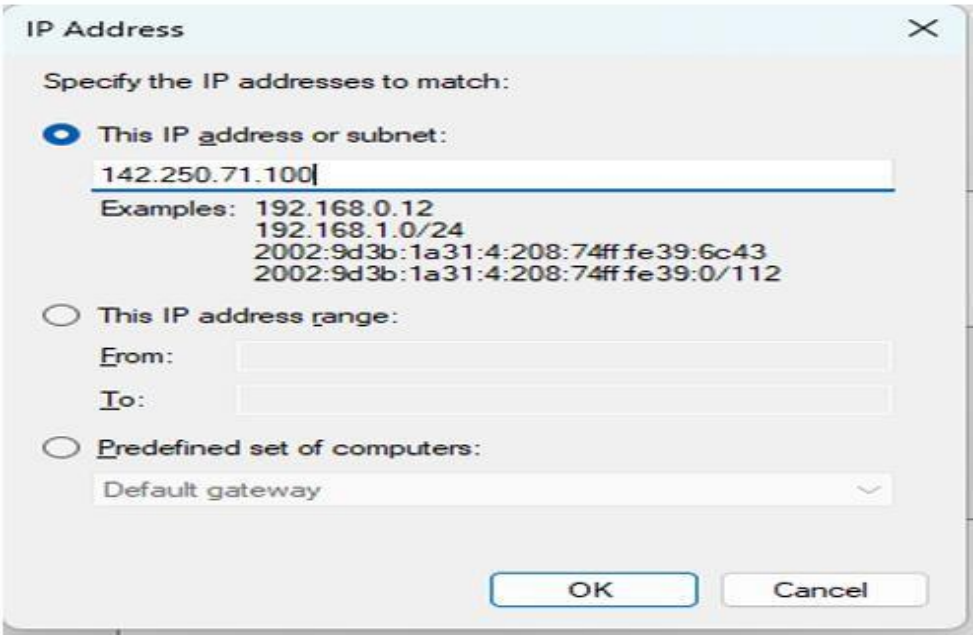


step 2 : select all program and click next .

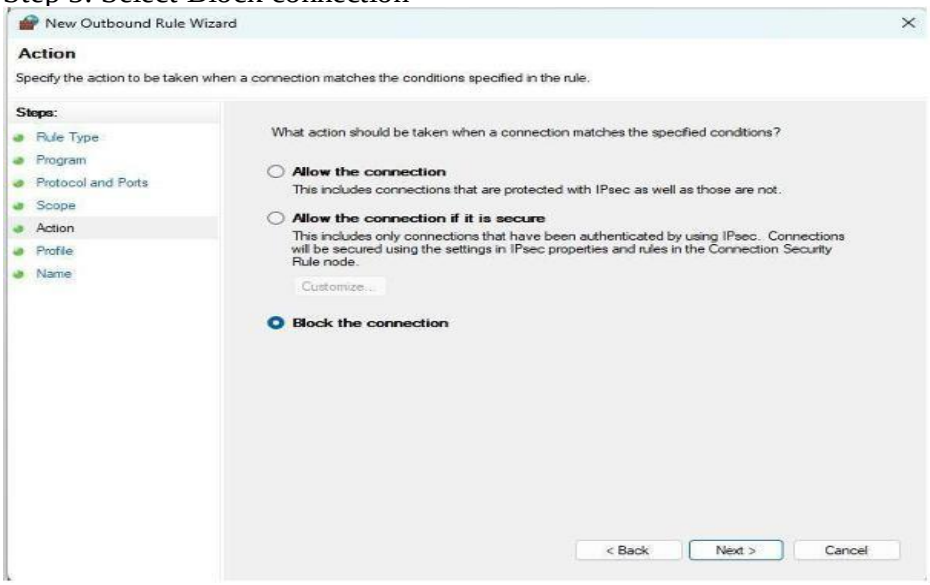


Step 4: Insert the IP addresses both IPv4 and IPv6.

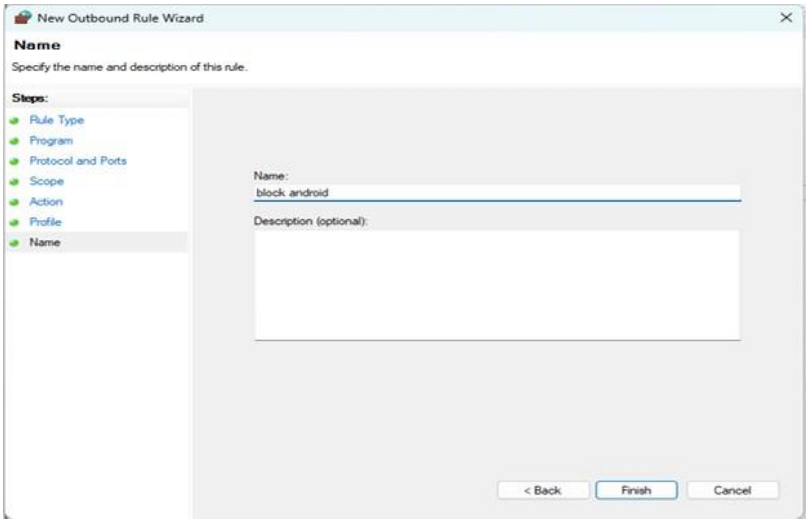




Step 5: Select Block connection



Step 6: Provide a suitable name and finish



Output: Now if we try to access the website www.android.com it would be blocked .

